B.Tech. CSE (IV YEAR - VII SEM) (2026-2027)

DEPARTMENT OF COMPUTER ENGINEERING & APPLICATIONS

GLA University

17km Stone, NH-19, Mathura-Delhi Road P.O. Chaumuhan,

Mathura 281406 (Uttar Pradesh)India

Project Title: Touch-Free Gesture Control Application Using AI Hans Tracking

TEAM LEADER	Sushil Kumar Yadav	2315510213
TEAM MEMBER 1	Manish Kumar	2315510118
TEAM MEMBER 2	Hirdesh Kumar	2315510087

Mentor Name: Mrs. Anushka Shukla

Signature:

Project Synopsis: Touch-Free Gesture Control Application Using AI Hand Tracking

Project synopsis for an AI-based touch-free application that detects hand gestures and maps them to computer control actions.

Cover

- **Project title:** Touch-Free Gesture Control Application Using AI Hand Tracking
- **Team members:** Sushil Kumar Yadav, Manish Kumar, Hirdesh Kumar
- **Institute / Course:** GLA University, Mathura / B.Tech CS(AIML & IOT)
- **Version:** v1.0
- **Date:** 10 Oct 2026
- **GitHub repository link:**

Revision history

Version	Date	Author	Change
v0.1	10 Oct 2026	Sushil Kumar Yadav	Initial draft prepared
v1.0	20 Oct 2026	Sushil Kumar Yadav, Manish Kumar, Hirdesh Kumar	Updated for touch-free AI hand-tracking project

1. Overview

- **Problem statement:** Traditional computer interaction depends on a keyboard, mouse, or touch screen. These devices may be inconvenient when a user's hands are occupied, when touch-free interaction is preferred, or when accessibility needs call for alternative input. A reliable way to recognize hand gestures and translate them into computer commands is therefore useful.
- **Goal:** Design and develop a touch-free gesture control application that uses AI-assisted hand tracking to detect hand landmarks in a live camera feed, recognize defined gestures, and trigger mapped computer actions in near real time.

- **Non-goals:** The initial version will not attempt to recognize every possible sign language, identify users biometrically, or replace all keyboard and mouse functions. It will not require cloud processing for its core gesture loop.
- **Value proposition:** Hands-free computer control through a webcam, visual feedback for recognized gestures, configurable gesture-to-action mapping, and local processing designed to reduce unnecessary data sharing.

2. Scope and Control

2.1 In-scope

- Camera input, hand detection and landmark tracking, gesture classification using defined landmark rules or a trained model, gesture-to-command mapping, visual status feedback, configurable controls, and basic functional testing.

2.2 Out-of-scope

- Full sign-language translation, multi-user identity recognition, cloud-based analytics, mobile application support, and control of safety-critical equipment are outside the initial scope.

2.3 Assumptions

- The user has a working webcam and grants camera permission. The environment has adequate lighting, and the supported gestures can be performed within the camera frame.
- The first release targets one user at a time and a limited, documented set of hand gestures. Recognition quality may vary with lighting, camera quality, distance, and occlusion.

2.4 Constraints

- The project is planned for an academic mini-project timeline and uses available laptops/webcams. Model performance and compatibility depend on hardware, software libraries, and camera conditions.

2.5 Dependencies

- Python runtime, webcam access, OpenCV for image capture and display, MediaPipe Hands or a comparable hand-landmark tracker, and optionally TensorFlow/Keras for a trained gesture classifier; PyAutoGUI or an equivalent approved interface may be used for mapped desktop actions.

2.6 Acceptance criteria and sign-off

- GIVEN the camera permission is granted WHEN a supported hand is visible in the frame THEN the application detects the hand and displays tracking feedback.
- GIVEN a supported gesture is held clearly WHEN the recognition confidence or rule condition is satisfied THEN the mapped action is triggered once or according to the configured repeat behavior.
- Sign-off: Project mentor reviews the demonstration, documented gesture mappings, test results, limitations, and safe-control behavior.

Sign-off table

Stakeholder	Role	Decision area	Signature/Approval	Date
Mrs. Anushka Shukla	Mentor	Scope, technical review, final acceptance	Pending	
Project Team	Team	Demo readiness and report submission	Pending	

3. Stakeholders and RACI

Activity	Responsible (R)	Accountable (A)	Consulted (C)	Informed (I)
Requirements	Sushil Kumar Yadav	All team members	Project mentor	Team
Design	Manish Kumar	All team members	Project mentor	Team
Implementation	Manish Kumar, Hirdesh Kumar	All team members	Project mentor	Team
Testing and documentation	Hirdesh Kumar	All team members	Project mentor	Team
Demonstration	All team members	All team members	Project mentor	Department

4. Team and Roles

Member	Role	Responsibilities	Key skills	Availability	Contact
Sushil Kumar Yadav	Project Coordinator	Requirements, planning, integration, review	Python, project documentation	8 hours/wk	sushil.yadav_cs.aiml23@gla.ac.in
Manish Kumar	AI/Hand-Tracking Developer	Camera pipeline, landmark tracking, gesture recognition	Python, OpenCV, MediaPipe, AI/ML	8 hours/wk	manish.kumar1_cs.aiml23@gla.c.in
Hirdesh Kumar	Testing and UI/Control Developer	Gesture mapping, status feedback, test cases, documentation	Python, testing, desktop-control integration	8 hours/wk	hirdesh.kumar_cs.aiml23@gla.ac.in

5. Week-wise Plan and Assignments

Week	Dates	Milestones	Sushil Kumar Yadav (Coordination)	Manish Kumar (AI/Tracking)	Hirdesh Kumar (Testing/UI)	Deliverables	Status
1	Week 1	Requirements and gesture set	Finalize scope and task plan	Review tracking options	Draft test plan and user flow	Requirements and gesture list	Planned
2	Week 2	Architecture and setup	Coordinate design review	Set up Python, OpenCV, MediaPipe	Prepare test environment	Architecture and setup notes	Planned
3	Week 3	Camera and hand tracking	Review progress and risks	Implement camera and landmark tracking	Test camera and tracking behavior	Working tracking prototype	Planned
4	Week 4	Gesture recognition	Review gesture definitions	Implement rule/model-based recognition	Create labeled test cases	Gesture recognition module	Planned
5	Week 5	Command mapping and feedback	Confirm scope and mapping	Integrate recognition output	Build status feedback and safe pause/stop	Integrated prototype	Planned
6	Week 6	Functional testing	Track defects and actions	Tune thresholds and stability checks	Run gesture and false-trigger tests	Test results and fixes	Planned
7	Week 7	Reliability and documentation	Review limitations and report	Optimize tracking/inference if needed	Complete diagrams and user guide	Final report draft	Planned

8	Week 8	Demo and submission	Coordinate final review	Demonstrate AI tracking pipeline	Present test results and documentation	Final demo and synopsis	Planned

6. Users and UX

6.1 Personas

- Student User: Wants to control selected computer functions without touching input devices; values ease of use and clear feedback.
- Accessibility-Focused User: Wants an alternative input method for supported actions; values predictable gestures, adjustable sensitivity, and a simple way to pause tracking.

6.2 Top user journeys

- User: Launch application → grant camera permission → position hand in frame → perform a supported gesture → see recognition feedback → mapped action is triggered. KPI: successful recognition rate on the defined test set.
- User: Open settings → review gesture mapping → enable or pause control → test a gesture → adjust sensitivity if supported. KPI: low false-trigger rate and clear recovery when tracking is lost.

6.3 User stories

- As a user, I want to perform a supported hand gesture to trigger a computer action so I can interact with selected functions without touching the keyboard or mouse. GIVEN tracking is active WHEN a valid gesture is recognized THEN the application shows the gesture and performs its mapped action.

6.4 Accessibility & localization

- Provide clear camera-on/camera-off status, a pause/stop control, readable on-screen feedback, adjustable thresholds where feasible, and instructions in English. Avoid relying on color alone to convey recognition status.

7. Market and Competitors

7.1 Competitor table

Approach	Product	Target users	Key features	Pricing	Strengths	Weaknesses	Project differentiator
Mouse/keyboard	Physical input devices	General computer users	Precise direct input	Hardware cost	Reliable and familiar	Requires touch and manual operation	Touch-free gesture alternative for selected actions
Voice control	Speech interface	Users needing hands-free input	Voice commands	Varies	Useful when hands are occupied	Noise and speech-recognition limitations	Visual hand-based control can work without spoken commands
Touchscreen	Touch input	Phone/tablet users	Direct manipulation	Device cost	Intuitive interaction	Requires physical contact	Webcam-based interaction on a compatible computer

7.2 Positioning

- Unique angle: A lightweight, camera-based hands-free interface that translates a small, explicit set of hand gestures into useful desktop actions.
- Measurable evaluation: Recognition accuracy, false-trigger rate, response latency, and tracking stability will be measured on a documented set of test gestures and recording conditions.

8. Objectives and Success Metrics

- O1 Gesture recognition: Measure accuracy for each supported gesture on a held-out test set. KPI: per-gesture and overall recognition rate.
- O2 Responsiveness: Measure time between a clearly performed gesture and its mapped action. KPI: median and 95th-percentile response time.
- O3 Reliability: Reduce unintended repeated actions through confidence thresholds, cooldowns, or stable-frame checks. KPI: false triggers per test minute.
- O4 Usability and safety: Ensure users can pause or exit control quickly and see when tracking is active. KPI: successful stop/pause tests and usability feedback.

9. Key Features

Feature	Description	Priority	Dependencies	Acceptance criteria
Camera and hand tracking	Capture frames and detect hand landmarks	Must	Webcam, OpenCV, MediaPipe	When a hand is visible, landmarks or tracking status are shown
Gesture recognition	Map landmarks to a limited set of gestures using rules or an optional trained model	Must	Hand landmarks, gesture definitions/model	Each supported gesture is evaluated against a labeled test set
Gesture-to-command mapping	Trigger a documented desktop action for a recognized gesture	Must	Recognition output, approved control library if used	Only configured gestures trigger actions; unknown gestures do not
Status and safety controls	Show tracking state and provide pause/stop behavior	Must	UI/control loop	User can stop tracking promptly and confirm that actions are disabled

Testing and evaluation	Measure accuracy, false triggers, and response time	Must	Test cases and recorded results	Report includes per-gesture results and known limitations

Gesture set and command mappings should be documented in the project report and updated to match the final implementation.

10. Architecture

10.1 High-level

- Client: Python desktop application with a live webcam preview and status/controls interface.
- Processing modules: Camera capture, hand landmark tracking, gesture recognition, command mapping, and feedback/control handler.
- Data: Hand landmark coordinates and transient frame data held in memory; optional labeled gesture samples/model files for training and testing.
- Libraries/integrations: OpenCV, MediaPipe Hands, optionally TensorFlow/Keras, and PyAutoGUI or an equivalent desktop-control interface.

10.2 API spec snapshot

Endpoint / Module	Method	Auth	Purpose	Input schema	Output schema	Codes / outcomes
camera.start	Local call	User permission	Initialize webcam	Camera index, resolution	Camera status	Success / camera error
hand_tracker.process_frame	Function call	No	Return hand landmarks	Image frame	Landmark coordinates or no detection	Detected / not detected

gesture_recognizer.predict	Function call	No	Recognize supported gesture	Landmarks/features	Gesture label, confidence if available	Label / unknown

10.3 Config and secrets

- Keep dependencies version-pinned where practical. Do not store camera frames by default. Keep model files and configuration local, and request camera permission before processing.

11. Data Design

11.1 Data dictionary

Entity	Field	Type	Null?	Allowed values	Source	Notes
GestureLabel	label	String	No	Documented gesture names	Project config	Unique gesture identifier
GestureMapping	gesture	String	No	Supported labels	Project config	Links gesture to action
GestureMapping	action	String	No	Allow-listed actions	Project config	Avoid destructive or sensitive actions
ModelConfiguration	threshold	Float	Yes	Configured range	Project config	Used if confidence threshold is implemented
TestResult	result_id	Integer/String	No	Unique test ID	Test run	Identifies one test result

11.2 Schemas and migrations

- Document the gesture-label schema, landmark feature representation, model version if used, and configuration settings. Keep training and test samples separate to avoid evaluation leakage.

11.3 Privacy, retention, backup/DR

- Process video locally where possible. Do not record or retain frames unless explicitly needed for training and the user has consented. Provide a visible tracking indicator and a clear stop control.

12. Technical Workflow Diagram

i. Application state diagram: Idle → Camera Initializing → Tracking → Gesture Recognized → Action Triggered → Tracking; any state can move to Paused or Error, then return to Idle.

ii. Sequence diagram: User performs gesture → webcam supplies frame → hand tracker returns landmarks → recognizer identifies gesture → command mapper checks confidence/cooldown → application executes the mapped action and displays feedback.

iii. Use cases: Start/stop camera, view hand landmarks, perform supported gesture, trigger mapped action, configure mapping/thresholds, pause controls, and review status.

iv. Data flow: Webcam frames → hand tracking → landmark features → gesture recognition → validation/cooldown → desktop command → visual feedback.

v. Data entities (if persisted): GestureLabel, GestureMapping, ModelConfiguration, and TestResult. Camera frames should not be persisted by default.

vi. Technical workflow: Initialize camera and libraries → capture frame → detect hand landmarks → calculate features → classify gesture → validate stability/confidence → execute mapped action → show feedback → repeat until stopped.

vii. Architecture: Camera input layer → hand-tracking layer → recognition layer → safety/command layer → UI feedback layer. Optional training workflow: labeled samples → preprocessing/features → model training → evaluation → exported model.

13. Quality: NFRs and Testing

13.1 Non-functional requirements

Metric	SLI	Target (SLO)	Measurement
Recognition accuracy	Correct predictions / labeled test samples	Establish baseline and improve through testing	Held-out gesture test set
Response latency	Time from stable gesture to action	Measure and document; optimize for responsive interaction	Timestamped test runs
False-trigger rate	Unintended actions per test minute	Minimize and report	Negative/idle gesture tests
Tracking reliability	Frames with valid hand tracking / eligible frames	Measure under documented conditions	Camera test sessions

13.2 Test plan

Area	Type	Tools	Owner	Coverage target	Exit criteria
Hand tracking	Functional	OpenCV, MediaPipe	Manish Kumar	Test all supported camera scenarios	Tracking behavior and failure cases documented
Gesture recognition	Unit/functional	Python test framework, labeled samples	Manish Kumar	Each supported gesture and unknown/ambiguous inputs	Results recorded; no unsupported action on unknown gesture
Command/UI	Integration	Manual tests; PyAutoGUI only if used	Hirdesh Kumar	All mapped actions and pause/stop behavior	All critical safety and control tests pass

13.3 Environments

- Development → local testing → controlled demonstration. Use a separate test mode before enabling desktop actions.

14. Security and Compliance

14.1 Threat model (STRIDE)

Asset	Threat	STRIDE	Impact	Likelihood	Mitigation	Owner
Camera stream	Unintended capture or retention	Information disclosure	Medium	Low/Medium	Local processing; no default recording; visible camera status	All members
Desktop actions	False gesture triggers action	Tampering	High	Medium	Allow-listed actions, confidence/stability checks, cooldown, pause/stop control	Hirdesh Kumar

14.2 AuthN/AuthZ

- Authentication is not required for a local single-user prototype. Camera access must be explicitly granted by the operating system and the user.
- Access control: The user starts and stops the application locally. Any desktop-control actions must be limited to the documented gesture mappings and should not perform destructive or sensitive operations.

14.3 Audit and logging

- Log only non-sensitive diagnostic events when needed, such as camera initialization failure, tracking status, and test outcomes. Avoid storing images or continuous video.

14.4 Compliance

- Academic prototype; follow institute rules and applicable privacy requirements. Explain camera use and avoid collecting biometric identity data.

15. Delivery and Operations

15.1 Release plan

- v1.0 demonstration after functional testing, documentation, and review of known limitations. Only enable desktop actions after confirming the gesture mapping in test mode.

15.2 CI/CD and rollback

- Development workflow: install dependencies → run unit/functional tests → validate recognition on held-out samples → package source and model/configuration → demonstrate with rollback/stop controls available.

15.3 Monitoring and alerting

Metric	Threshold	Alert to	Runbook
Tracking/response delay	Repeated delay beyond project target	Project team	Check camera resolution, FPS, CPU load, and inference time
False-trigger rate	Unexpected actions during idle/negative tests	Project team	Disable actions, review thresholds and gesture definitions, rerun tests

15.4 Runbooks

- Recognition delay: check frame rate, camera resolution, landmark tracking load, and inference time; reduce resolution or simplify the gesture set if needed.
- False-trigger spike: pause desktop control, review lighting and thresholds, adjust stability/cooldown settings, and rerun the test set before enabling actions.

15.5 Communication plan

- Team check-ins at agreed intervals, shared task tracking, code review before integration, and a final demonstration with a written test report.

16. Risks and Mitigations

16.1 Risk heatmap

Risk	Probability	Impact	Score	Mitigation	Owner	Status
Poor lighting or occlusion	Medium	High	12	Document lighting guidance, test varied positions, use stable-frame checks	Manish Kumar	Open
False-triggered actions	Medium	High	12	Allow-list commands, cooldown, visible status, pause/stop control	Hirdesh Kumar	Open
Hardware/library compatibility	Medium	Medium	9	Pin dependencies, test on target laptop, document setup steps	Sushil Kumar Yadav	Open

17. Research and Evaluation

- Research review: compare webcam-based hand tracking approaches, landmark-based gesture rules, and optional trained classifiers for their accuracy, latency, setup complexity, and privacy implications.
- Evaluation plan: test each supported gesture under varied lighting, hand positions, and distances; record true detections, missed detections, false triggers, and response latency.
- Limitations: performance depends on camera quality, lighting, occlusion, hand orientation, and the chosen gesture set. The prototype is not intended for safety-critical control or unrestricted sign-language interpretation.

18. Appendices

- Glossary: AI, CV, FPS, hand landmarks, gesture classification, confidence threshold, false trigger, latency.
- References: Python documentation; OpenCV documentation; MediaPipe Hand Landmarker documentation; TensorFlow/Keras documentation (if used); PyAutoGUI documentation (if used).